

Program 6: Seprability and Non linear separability

Aim :

To implement AND, OR, and XOR logic gates using Artificial Neural Networks and analyze linear separability and non-linear separability of problems.

Theory :

A problem is linearly separable if we can draw a single straight line (in 2D) (or hyperplane in higher dimensions) to separate classes.

Mathematically, a perceptron learns:

$$y=w_1x_1+w_2x_2+b$$

2. AND Gate (Linearly Separable)

x1	x2	Output
0	0	0
0	1	0
1	0	0
1	1	1

✓ A single line can separate (1,1) from others

3. OR Gate (Linearly Separable)

x1	x2	Output
0	0	0
0	1	1
1	0	1
1	1	1

✓ Also separable using a line

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. Dataset
# -----
X = np.array([[0,0],[0,1],[1,0],[1,1]])

Y_or = np.array([[0],[1],[1],[1]])
Y_and = np.array([[0],[0],[0],[1]])
Y_xor = np.array([[0],[1],[1],[0]])

# -----
# 2. Perceptron Training (OR, AND)
# -----
def train_perceptron(X, Y, lr=0.1, epochs=10):
    np.random.seed(42)
    W = np.random.randn(X.shape[1], 1)
    b = np.random.randn(1)

    for _ in range(epochs):
        for x, y in zip(X, Y):
            z = np.dot(x, W) + b
            y_pred = 1 if z >= 0 else 0

            error = y - y_pred

            W += lr * error * x.reshape(-1,1)
            b += lr * error

    return W, b

W_or, b_or = train_perceptron(X, Y_or)
W_and, b_and = train_perceptron(X, Y_and)

# -----
# 3. XOR using Neural Network
# -----
np.random.seed(42)

W1 = np.random.randn(2,2)
b1 = np.zeros((1,2))
```

```

W2 = np.random.randn(2,1)
b2 = np.zeros((1,1))

lr = 0.1
epochs = 10000

def sigmoid(x):
    return 1/(1+np.exp(-x))

def sigmoid_deriv(x):
    return x*(1-x)

loss_history = []

for _ in range(epochs):

    # Forward
    Z1 = X.dot(W1) + b1
    A1 = sigmoid(Z1)

    Z2 = A1.dot(W2) + b2
    A2 = sigmoid(Z2)

    # Loss
    loss = np.mean((Y_xor - A2)**2)
    loss_history.append(loss)

    # Backprop
    dA2 = A2 - Y_xor
    dZ2 = dA2 * sigmoid_deriv(A2)

    dW2 = A1.T.dot(dZ2) / X.shape[0]
    db2 = np.sum(dZ2, axis=0, keepdims=True) / X.shape[0]

    dA1 = dZ2.dot(W2.T)
    dZ1 = dA1 * sigmoid_deriv(A1)

    dW1 = X.T.dot(dZ1) / X.shape[0]
    db1 = np.sum(dZ1, axis=0, keepdims=True) / X.shape[0]

    # Update
    W2 -= lr * dW2
    b2 -= lr * db2
    W1 -= lr * dW1

```

```

b1 -= lr * db1

# -----
# 4. Decision Boundary Plot
# -----
def plot_boundary(X, Y, W=None, b=None, NN=False,
                  W1=None, b1=None, W2=None, b2=None,
                  ax=None, title=''):

    xx, yy = np.meshgrid(np.linspace(-0.2,1.2,200),
                          np.linspace(-0.2,1.2,200))

    grid = np.c_[xx.ravel(), yy.ravel()]

    if NN:
        Z1 = sigmoid(grid.dot(W1) + b1)
        Z2 = sigmoid(Z1.dot(W2) + b2)
        Z = (Z2 > 0.5).astype(int)
    else:
        Z = (grid.dot(W) + b >= 0).astype(int)

    Z = Z.reshape(xx.shape)

    ax.contourf(xx, yy, Z, alpha=0.3, cmap='bwr')
    ax.scatter(X[:,0], X[:,1], c=Y.flatten(),
               s=100, cmap='bwr', edgecolor='k')

    ax.set_title(title)
    ax.set_xlim(-0.2,1.2)
    ax.set_ylim(-0.2,1.2)
    ax.set_xlabel('Input 1')
    ax.set_ylabel('Input 2')

# -----
# 5. Plot Results
# -----
fig, axes = plt.subplots(1,3, figsize=(15,5))

plot_boundary(X, Y_or, W=W_or, b=b_or, ax=axes[0], title='OR Gate (Linear)')
plot_boundary(X, Y_and, W=W_and, b=b_and, ax=axes[1], title='AND Gate (Linear)')
plot_boundary(X, Y_xor, NN=True, W1=W1, b1=b1, W2=W2, b2=b2,
              ax=axes[2], title='XOR Gate (Nonlinear)')

plt.tight_layout()
plt.show()

```

Output :

